



Design Patterns

Factory Method vs Abstract Factory

Both hide object creation.

Both follow Open/Closed.

So what actually changes?

Java

Creational patterns

6 slides

Let's find out — slide by slide.

Why do factory patterns exist?

1

Direct instantiation couples your code

`new HyundaiCar()` in client means client is tightly bound to that concrete class.

2

Swapping variants breaks everything

Want Maruti instead? Change every `new` call in client. That violates Open/Closed.

3

Factory patterns delegate creation

Client talks to an interface only. Swap the factory → swap the product. Zero client changes.

Both patterns solve this. Their only difference: how many products they manage.

One factory, one product

The showroom analogy: walk in and say "give me a Hyundai car." One product returned.

```
// FactoryMethod package
Factory f = new HyundaiFactory();
Car car = f.createCar();    // one product

// Swap to Maruti — zero client changes
Factory f = new MarutiFactory();
Car car = f.createCar();
```



1 method on the interface

createCar() — that's it



Each factory = one variant

HyundaiFactory → HyundaiCar



Client codes against Car

Never touches HyundaiCar directly

One factory, a whole ecosystem

The manufacturer analogy: building a full vehicle — Engine, Dashboard, and Tyres must all belong to the same brand family.

```
// AbstractFactory package
CarFactory f = new HyundaiFactory();

Engine e = f.createEngine();    // HyundaiEngine
DashBoard d = f.createDashBoard(); // HyundaiDash
Tyres t = f.createTyres();      // HyundaiTyres

// guaranteed: all from same family
```



3 methods on the interface
createEngine, createDashBoard,
createTyres



Never mix brands accidentally
Can't get Hyundai tyres with a Maruti
engine



**Swap factory = whole ecosystem
swaps**
Atomically and safely

Side by side — what actually changes



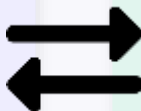
Factory Method

Creates 1 product type

Interface has 1 method

Use when: variants of one thing

e.g. Car → HyundaiCar / MarutiCar



Abstract Factory

Creates a product family

Interface has N methods

Use when: related objects must match

e.g. Engine + DashBoard + Tyres together

Ask yourself: "Am I creating one thing, or multiple things that must belong together?" That question picks the pattern.



Key takeaways

- 1 Factory Method — delegate creation of one product. Swap factory to swap variant.
- 2 Abstract Factory — delegate creation of a product family. Guarantees consistency across related objects.
- 3 Both follow Open/Closed — add a new brand by adding a new factory class, never touching existing code.